AI Changes Both Sides of Software Engineering

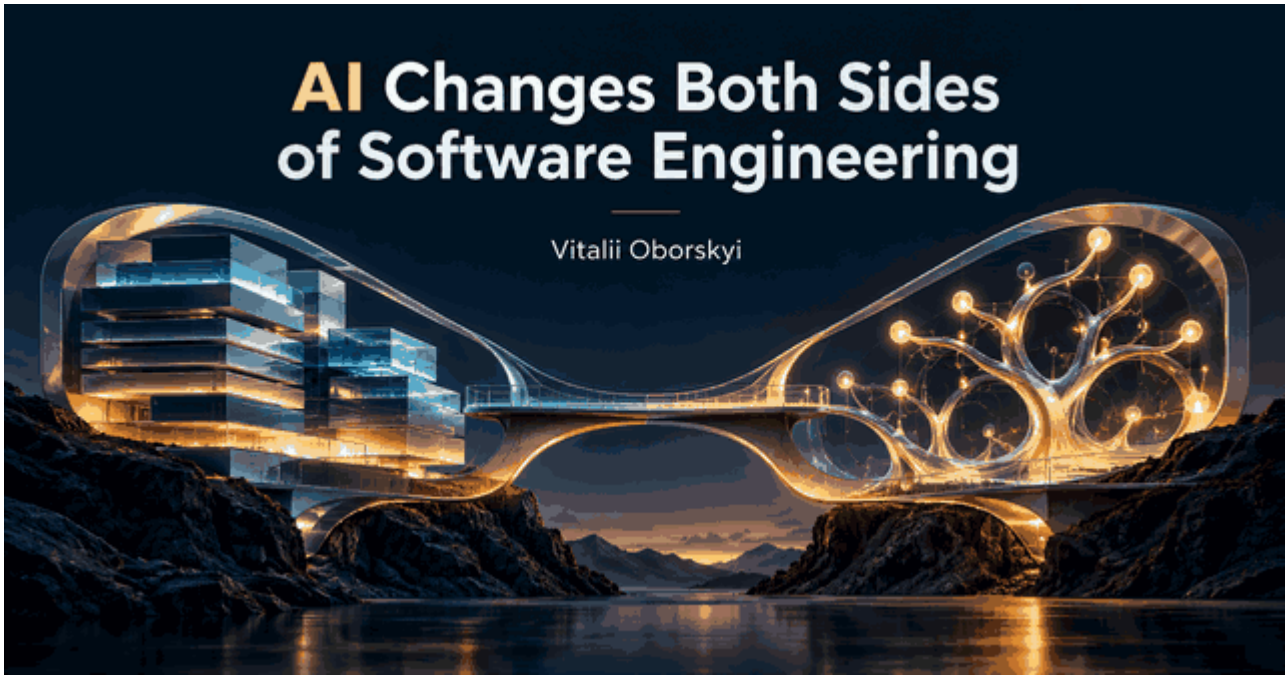
Vitalii Oborskyi

AI changes how we build software and what the software we build is responsible for doing. Faster delivery, shared understanding, useful outcomes and operational control have to evolve together.

---

1. When ideas outlive their authors
2. We accelerated code generation. The rest of the work remained.
3. Who understands the system when it breaks?
The documentation paradox
Promising methods still need evidence
4. Faster delivery to what end?
Your customer also has limited capacity
5. When judgment becomes part of the product
A high average can hide an unacceptable result
Control has to reach the consequence
6. Build for success. Prepare to act if you are wrong.
Sources and further reading

We can delegate more work. We still have to own the consequences.



We can delegate more work. We still have to own the consequences. AI-generated editorial illustration directed for this article; built-in ImageGen, 15 October 2026.

While preparing my PMDay talk, I realised that the presentation needed an article. The slides brought together two changes we often discuss separately: how AI changes the way we build software, and how it changes the responsibilities of the software we build.

Both lead to a question that becomes more important as the tools improve: **what must we remain capable of doing after we delegate the work?**



PMDay presentation, v35 review edition, 15 slides. Recording in Ukrainian. Vitalii Oborskyi. Preview rendered from the pinned v35 PDF; presentation credits remain in the source deck.

> **The presentation behind this article**
>
> Open the presentation — PDF, v35 review edition · Watch the PMDay talk — in Ukrainian
>
> The article develops the argument behind the slides and adds subsequent examples. It can be read independently.

1. When ideas outlive their authors

Humanity keeps changing. The reach of a change depends partly on where it enters our lives. Improving a particular tool affects a particular activity. Changing how people preserve, circulate or work with ideas can reach almost every institution built around them.

Writing allowed an idea to survive the person who expressed it. Printing changed the scale and cost of its circulation. A thought could escape an isolated conversation, travel across borders, wait for another generation and become part of somebody else's discovery.

How many scientific breakthroughs, political movements, conflicts and revolutions became possible because ideas could travel through time and space?

We cannot reconstruct that entire alternative history. We can study particular effects, such as printing and economic change or printing and the Reformation. Their significance extends far beyond producing more books.

LLMs expand what we can delegate again: interpreting a request, comparing alternatives, proposing a design, drafting an explanation. I call this *delegating thinking* in a practical sense. It makes no claim about consciousness, and earlier computers already performed substantial cognitive work. What is expanding is the range and accessibility of that delegation.

I expect its consequences to reach across society on a comparable scale. That is an expectation, not a historical fact we can establish in advance.

Software engineering gives us an immediate place to examine the change. AI participates in producing software. Model judgment also becomes part of what some software does while running.

These are independent dimensions. An AI coding agent can build an ordinary application. A conventionally staffed team can build a product that depends on an LLM. Many teams will encounter both.

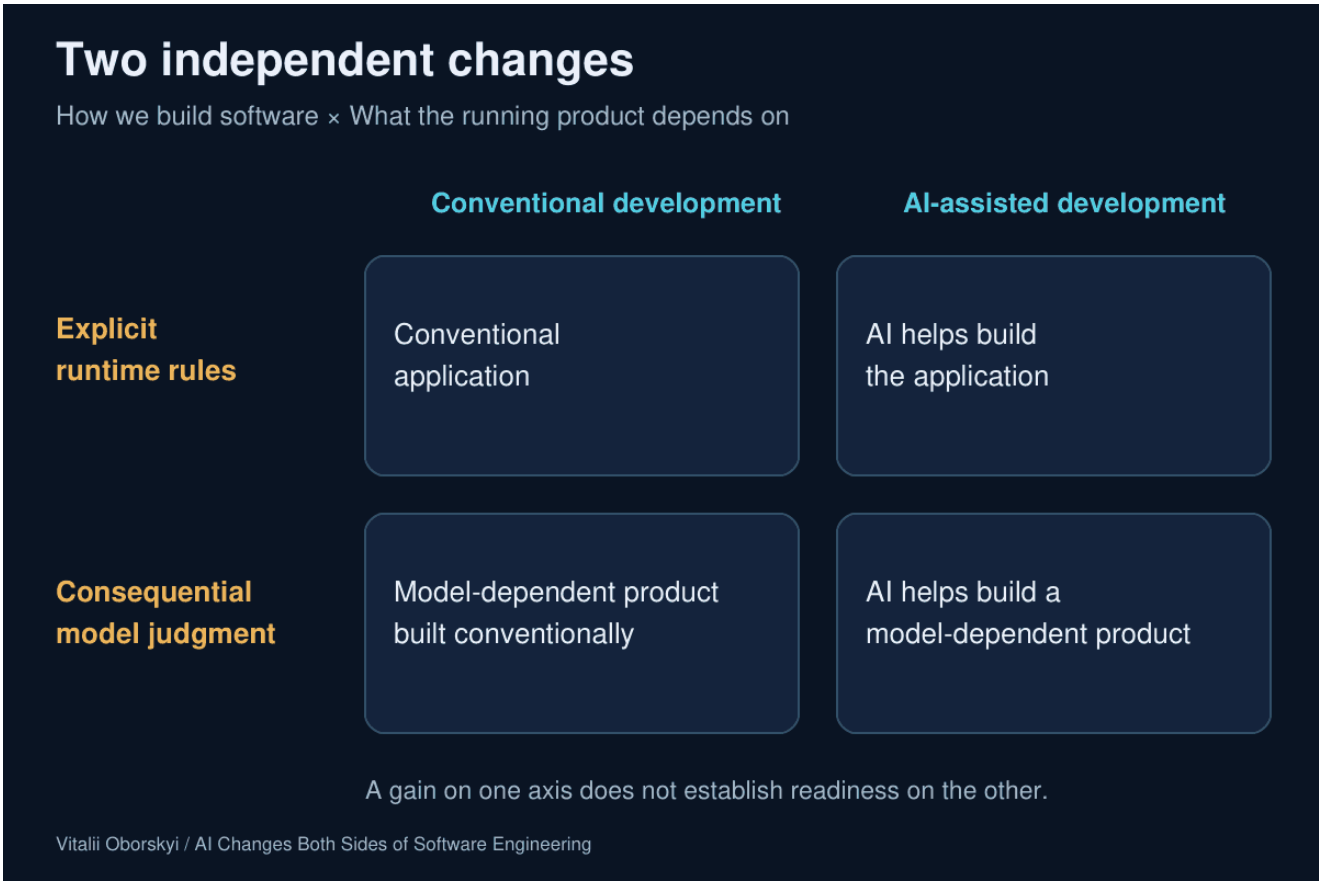


Figure 1. How software is built and what it depends on at runtime are independent questions. Original diagram for this article, based on the PMDay HOW/WHAT distinction.

2. We accelerated code generation. The rest of the work remained.

Requirements, design, implementation, review, verification, integration and operation form one delivery system. Changing the capacity of one activity changes the balance between them.

Consider a simple illustration. A team can produce, review and integrate eight comparable changes a week. Generation capacity rises to one hundred. Review and integration remain at eight.

If all one hundred changes enter the queue, unfinished work grows by ninety-two a week.

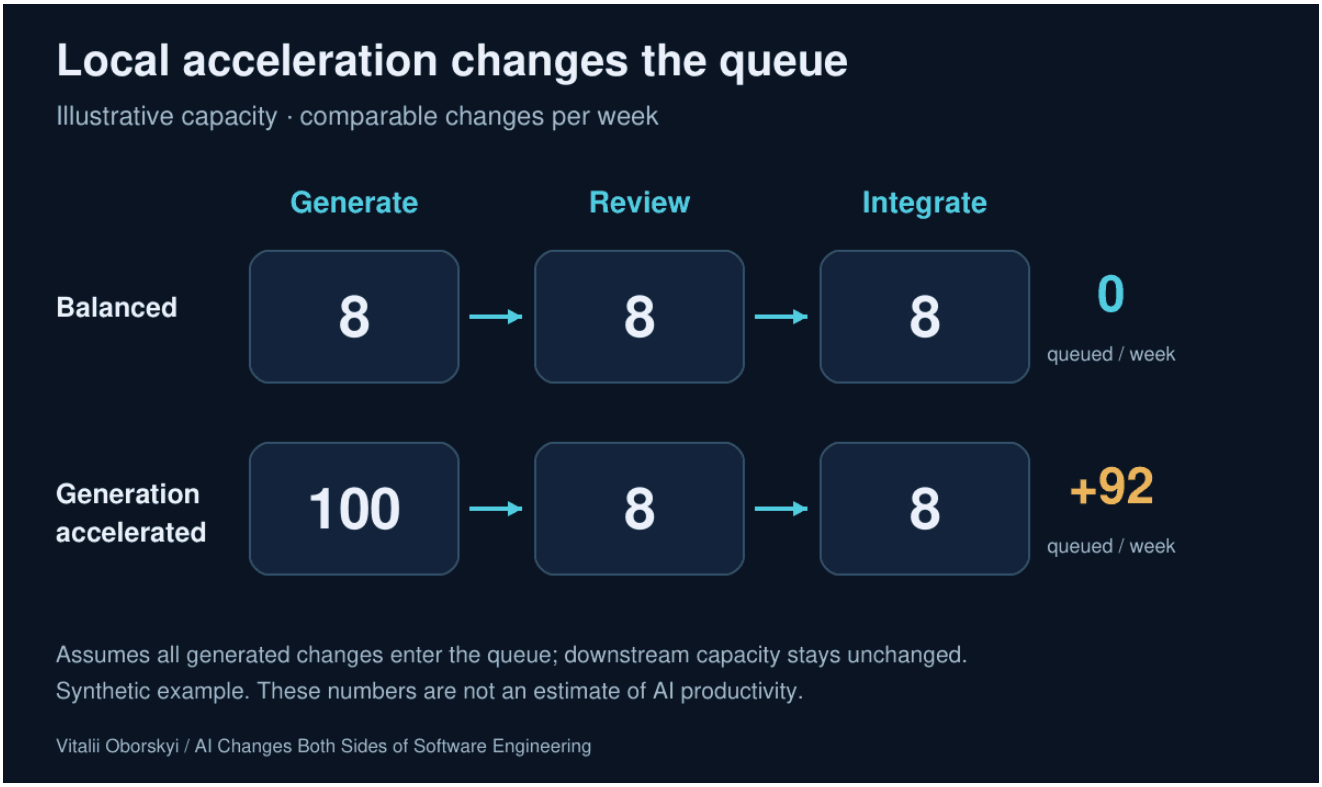


Figure 2. Local acceleration can increase unfinished work. Synthetic capacities; downstream capacity is unchanged and all generated work is admitted. Original explanatory diagram; no measured AI productivity result is represented.

Review did not suddenly become unnecessary. We changed how much work arrived there.

Yet the language around the problem matters. Anthropic's Code Review announcement identifies review as a bottleneck after code output increased. Microsoft's Jay Parikh uses the headings “Don’t let code review become a bottleneck” and “Code is disposable” in his new rules for builders.

The pressure is real. So is the opportunity to improve review. But identifying a constraint does not tell us which functions we can safely remove. We may need better tools, smaller changes, different architecture, more verification capacity or less work entering the system.

A queue can expose an old constraint. When we accelerate generation without rebalancing the lifecycle, we also help create the imbalance we then describe as a human limitation.

Vendors recognise the wider problem too. Anthropic's SDLC playbook explicitly addresses lifecycle changes and retained control objectives. The useful question is which replacement practices have demonstrated the outcomes we need.

Its October account of Scion is instructive. Anthropic reports growth from about 2,500 daily merges in May to over 30,000 in September, and checkouts roughly one hundred times faster after redesigning source control.

Those are substantial infrastructure results. Checkout speed is not end-to-end delivery speed or customer value. The company describes an unusual workload and says Git remains appropriate for most repositories.

Crucially, engineers retained architectural understanding and reviewed every change for intent. The account also describes extensive simulations and fuzzing. This is evidence of an attempt to rebalance the supporting system. Adopting its reduced emphasis on reading every line without the accompanying expertise and verification would be a different experiment.

The research also demands care about what we count. The September revision of *Writing Code vs. Shipping Products*, as reconciled in the presentation's evidence register, reports much larger changes in code activity than in releases. Those measures do not form a conversion rate from code to customer benefit.

DORA 2025 associates greater AI adoption with both higher delivery throughput and greater delivery instability. This is survey evidence, not proof that AI caused either outcome.

We need to measure the system we actually operate: elapsed time, useful outcomes, rework, stability, understanding and recovery. Counting generated code cannot answer those questions.

3. Who understands the system when it breaks?

Code review has two important outputs.

One is inspection: finding defects, inconsistencies and violated constraints. Another is shared understanding: somebody beyond the original author learns what changed, why it changed and how it fits into the system.

This broader role is documented in Bacchelli and Bird's study of modern code review. Their observations of Microsoft teams identified knowledge transfer and team awareness alongside defect detection.

Review can therefore function as an ownership gate as well as a quality gate.

That does not make every human review effective. A ceremonial approval may produce neither quality nor understanding. It does mean that replacing review requires us to account for both functions.

An automated reviewer might detect more defects while teaching the on-call team very little about the system. We could improve one outcome and lose another without noticing until an incident.

Responsibility does not disappear when understanding disappears. The team may simply become less capable of fulfilling it.

The documentation paradox

Generated documentation is a promising way to reduce the cost of understanding. Cognition's DeepWiki, for example, is explicitly presented as a tool for navigating and understanding repositories.

But consider the proposed shortcut. We cannot keep up with reviewing generated code, partly because understanding it takes time. So we ask an LLM to explain it.

Now we have an implementation and an explanation. What establishes that they agree?

The explanation can omit an exception, describe intended rather than actual behaviour, or repeat an assumption that was already wrong in the code. Readable prose can make an error easier to accept.

We have moved the understanding problem to another representation. We may have made it cheaper to solve. We have not automatically solved it.

To trust the explanation, we need evidence connecting its claims to the implementation and observed behaviour. Even a faithful explanation does not establish that the implementation satisfies the requirement.

This need not mean manually checking every sentence against every line. Version-specific code links, executable examples, observed traces, independently grounded tests and focused checks of critical claims can make verification cheaper.

The practical test is whether engineers can use the documentation to predict important behaviour and diagnose an unfamiliar failure—and whether their answers survive verification.

Promising methods still need evidence

Spec-driven development can clarify intent before implementation. Regeneration can make bounded components cheaper to replace. AI review can broaden inspection.

Specifications, documentation and replaceable components have long engineering histories. The hypothesis is more specific: that a particular combination of agents and artifacts can replace a particular human activity while preserving its necessary outcomes.

A specification may omit a critical exception. Generated code and tests may share the same mistaken assumption. Disposable implementation may still operate on persistent data and support obligations that cannot be discarded with it.

We should test these approaches seriously. Their benefits may be substantial.

We should also test the incident they are expected to survive.

Imagine a critical workflow failing after weeks of rapid delivery. Transactions stop. The assistant produces plausible explanations and successive patches. None restores the service. Nobody is confident whether a proposed rewrite will damage existing data.

This is a hypothetical scenario. It is also a reasonable design exercise.

Who can contain the impact? Can the team restore a known-good state? What if a migration makes rollback unsafe? Can it investigate if the coding service is unavailable? Who establishes that recovery worked?

Adopting a coding tool does not, by itself, transfer operation of your service to its provider. Your team still needs a credible answer for customers when the service fails.

AI may be part of that answer. It may diagnose and repair faster than people could. The operating plan still needs to address the case where its preferred repair path fails.

Ownership does not require memorising every line. It requires a demonstrated ability to investigate, challenge and change the system at the level where its consequences matter.

4. Faster delivery to what end?

Suppose we solve the capacity problem. The organisation can now bring twenty-five times as many ideas to users.

This is a thought experiment, not a productivity estimate from the NBER paper.

Have we built a better product organisation?

In a slower system, an idea often has to survive conversations with business stakeholders, engineers and operations. Someone asks whether customers need it, whether its assumptions make sense, or whether the operating cost is justified. Some weak ideas die before implementation.

Cheap implementation can make it easier to bypass that challenge. If we also weaken selection, we can deliver more poorly examined ideas with remarkable efficiency.

But slow delivery is an unreliable filter. Good ideas die in queues too. A working prototype may answer a question more cheaply than another meeting.

We should preserve useful challenge while removing avoidable waiting. Research on online experimentation shows why: measured customer behaviour can challenge confident predictions about which changes will help.

Extra capacity can fund more experiments and better decisions, including decisions to stop. Shipping broadly is only one possible outcome of learning.

Your customer also has limited capacity

The human factor remains after deployment.

People have to discover a feature, understand when it matters, learn a changed interaction and sometimes reorganise work around it. Even individually useful features can impose a collective burden.

Feature-fatigue research found that people can favour capability before use and place greater weight on usability afterwards. That does not give us a universal limit on release frequency. It does challenge the assumption that adding attractive capabilities necessarily improves the experience.

Imagine receiving twenty-five times as many useful features. Would you have twenty-five times as much time to learn them? Would your colleagues? Would the organisation coordinating your work?

The advantage depends on what changes. Faster fixes, better accessibility, lower latency and fewer unnecessary steps can improve a product without increasing the user's learning burden. Targeted rollout can keep irrelevant changes out of people's way.

The user benefits from a better outcome, not from our ability to generate a larger release note.

For delivery leadership, the objective needs to include customer benefit, adoption burden, stability, recovery and total operating cost alongside speed. Otherwise, we risk making the delivery dashboard look healthier while making the product harder to live with.

5. When judgment becomes part of the product

So far, AI has helped produce software. The second shift occurs when the running product depends on model judgment to fulfil a responsibility.

Consider an assistant helping someone recover access to an account. It interprets their situation, retrieves guidance and recommends what to do. Its answer can have consequences even if it cannot change the account directly.

In Uncertainty Architecture, I call this a *Thinking System*: a system in which a consequential runtime responsibility depends partly on probabilistic model judgment.

The term identifies an engineering property. It does not establish consciousness or adequate control.

Several differently worded answers may all be acceptable. An invented recovery step or restricted disclosure is a materially different outcome.

Requirements have to express that distinction:

Behaviour	Requirement for the illustrative assistant
Acceptable variation	Explain authorised recovery steps clearly, preserving required warnings
Clarification or escalation	Ask for missing information or refer conflicting guidance to an authorised person
Prohibited outcome	Disclose restricted information or perform an unauthorised access change

Identity, permissions and transaction rules still need exact enforcement. A model's involvement does not turn them into approximate obligations.

A high average can hide an unacceptable result

The presentation uses a synthetic example: 200 evaluated responses, of which 196 are acceptable, three contain incorrect recovery instructions, and one discloses restricted information.

We could present that as 98% acceptable.

Under an agreed rule prohibiting that disclosure, this candidate still fails the release condition. The average cannot cancel the consequence.

A curated Golden Set helps exercise known requirements and difficult cases. It does not automatically represent production traffic. Estimating failure frequency requires suitable sampling and explicit uncertainty. Testing varied behaviour, as illustrated by CheckList, addresses a different question from repeatedly sampling one convenient prompt.

The evaluator can also be wrong. We need to know what it checked, against which criteria, using which system version, and where its judgments are unreliable.

This changes the delivery conversation. Before implementation, we need enough clarity about responsibility, constraints and the evaluation plan. Completion requires evidence against those commitments. Release requires an authorised decision about a defined operating scope and its remaining risks.

A successful demo cannot stand in for all three.

Control has to reach the consequence

Pre-release evaluation cannot cover every situation the system will encounter. The running system needs ways to constrain actions, recognise problems and support effective intervention.

For the access assistant, that means restricting retrieval to authorised information, checking consequential claims, enforcing permissions before action and providing a usable fallback when required conditions fail.

Afterwards, we need to observe outcomes and control health. Someone with appropriate authority must be able to narrow the task, disable a tool, suspend a path or change the configuration—and verify that the intervention worked.



Figure 3. Control before and after consequence. Semantic checking is fallible; permissions constrain authority. Later intervention cannot necessarily reverse earlier consequences. Original diagram for this article, informed by PMDay slide 12 and Uncertainty Architecture.

A dashboard can expose a problem without providing the ability to stop it. “Human in the loop” can describe a person who lacks the time, information or authority to intervene.

Some consequences are irreversible. You cannot undisclose a credential with a rollback.

The appropriate controls depend on consequence, authority and reversibility. Drafting a suggestion for a qualified reviewer and autonomously changing account access require different arrangements. This lifecycle perspective is also reflected in the NIST AI Risk Management Framework.

Control consumes time, money and human capacity. Those costs belong in the product decision. If we cannot provide effective control within the required window, narrowing the responsibility is a valid engineering choice.

6. Build for success. Prepare to act if you are wrong.

The two shifts meet in one operating model.

We may generate software faster while building products that need more domain evaluation, clearer authority boundaries and stronger operational response. A plan based only on implementation capacity misses both changes.

This is where delivery leadership matters: connecting product intent, engineering constraints, evidence, release decisions and operational capability. Making sure knowledge reaches the people who will maintain the system. Giving that work owners, capacity and time.

It does not require inventing a new department. It requires making the responsibilities real.

Start with one bounded workflow. Define the useful outcome, the authority delegated and the outcomes that must be prevented. Establish a credible baseline. Measure customer benefit, elapsed time, human effort, rework and operating cost.

Then rehearse failure.

Can the team detect a problem, contain its consequences and establish recovery if the preferred automated path stops working? Can it explain the evidence it is relying on? Can it change course without depending on the very assumption that failed?

Those questions belong in an adoption trial alongside the productivity demonstration.

Use the recovered capacity deliberately. Invest it in experiments, maintenance, simpler workflows or faster delivery of a proven need. More capacity gives us choices; it does not oblige us to expand the feature queue.

I believe the optimistic future is plausible. AI could make useful software cheaper to build, easier to understand and safer to operate. It could help us test more thoroughly, repair faster and address problems we previously could not afford to solve.

But we have to run organisations and serve customers while those possibilities are still being tested.

A workflow may improve defect detection while eroding shared understanding. A generated explanation may sound convincing while describing the wrong behaviour. A repair agent may fail when it is most needed. A stream of excellent features may still make a customer's life more complicated.

We need the ability to discover that, limit the consequences and respond.

Writing let ideas outlive their authors. We are now expanding how much work with those ideas we can delegate. The choices about what is worth building, what evidence is sufficient and what consequences we accept remain ours.

Build for the future you believe in. Keep the ability to act if that belief is wrong.

Sources and further reading

- **Presentation and framework:** PMDay slide source, version-specific evidence register, Uncertainty Architecture and The Subprime Code Crisis.
- **Historical mechanisms:** Dittmar, Information Technology and Economic Change, 2011; Rubin, Printing and Protestants, 2014.
- **Delivery evidence:** Demirer, Musolff and Yang, Writing Code vs. Shipping Products, September 2026 revision; DORA 2025. The presentation's evidence register retains the additional METR, GitClear and repository-analysis sources and their limitations.
- **Vendor proposals and reported practice:** Anthropic, Code Review, March 2026; Jay Parikh/Microsoft, New rules for builders, June 2026; Anthropic, AI-native SDLC playbook, August 2026, and Redesigning source control, October 2026; GitHub, Spec Kit introduction, 2025; Cognition, DeepWiki, 2025.
- **Review and product decisions:** Bacchelli and Bird, Expectations, Outcomes, and Challenges of Modern Code Review, 2013; Thompson, Hamilton and Rust, Feature Fatigue, 2005; Kohavi and colleagues, Online Experimentation at Microsoft, 2009.
- **Evaluation and control:** Ribeiro and colleagues, Beyond Accuracy: Behavioral Testing of NLP Models with CheckList, 2020; NIST AI RMF, 2023, and Generative AI Profile, 2024; UA's control-loop anatomy.